

HYPERSKILL: 基于超图结构化技能记忆的自我进化大语言模型智能体

Ruiyao Xu¹ Tiankai Yang² Wei-Chieh Huang³

¹Northwestern University

²University of Southern California

³University of Illinois at Chicago

摘要

随着智能体任务复杂度的提升，大语言模型智能体日益依赖经验记忆来跨任务复用程序性知识。有效的记忆设计必须同时解决存储内容、结构化与检索方式以及记忆演化机制三个问题。现有系统仅部分解决这些问题：它们将轨迹、洞见或工作流程作为孤立条目存储，丢弃了子任务与可复用技能之间的组合关系；采用忽略关系信号的扁平嵌入相似度进行检索；且未利用关系结构来维护记忆。本文提出 HYPERSKILL，一个基于超图的记忆框架，联合改进上述三个方面。HYPERSKILL 将记忆表示为包含子任务步骤和可复用技能两类节点的超图，其中每条超边连接来自单条轨迹的子任务与技能。双路径检索同时查询子任务层和轨迹层，并根据技能在检索到的轨迹中的共现频率进行排序。周期性的结构感知维护通过质量加权传播剪除低效用节点并合并冗余技能。在 xBench、GAIA 和 WebWalkerQA 基准上，使用 GPT-4o 和 Qwen3-30B-A3B 模型，HYPERSKILL 优于十种记忆基线方法，在 GAIA 上提升高达 +11.51，在 WebWalkerQA 上提升高达 +11.18。¹

1 引言

大语言模型智能体的快速发展 (Liu 等, 2025b; Yao 等, 2022; Shinn 等, 2023) 显著拓宽了自主系统可处理的任务范围，涵盖长程网页导航 (Chen 等, 2025; Gur 等, 2024; Mialon 等, 2023)、交互式计算机操作 (Yang 等, 2024; Xie 等, 2024) 以及多步科学推理 (Ghafarollahi 和 Buehler, 2025; Ren 等, 2025)。随着任务复杂度的提升，

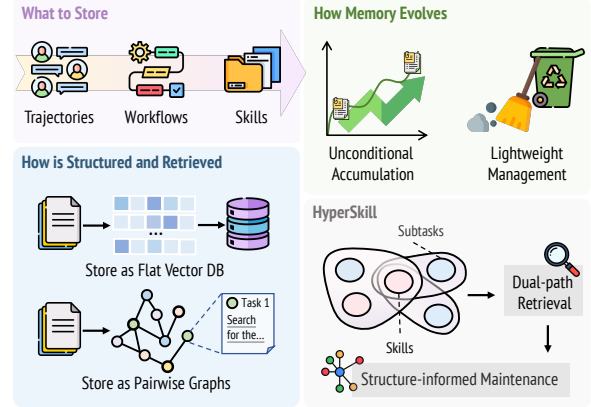


图 1：经验型智能体记忆的三个设计维度。HYPERSKILL 通过子任务与技能上的超边、双路径检索以及结构感知维护，统一了存储内容、记忆的结构化与检索方式以及记忆演化机制。

复杂度和智能体在连续的情节流中运行，单次交互很少足以可靠地解决问题。智能体记忆系统 (Huang et al., 2026; Hu et al., 2025) 通过存储和检索超出单个上下文窗口的信息来解决这一问题。其中，经验记忆 (Wang et al., 2025b; Ouyang et al., 2026) 尤为重要：它并非编码外部事实，而是将智能体自身的过往轨迹提炼为可复用的知识，从而实现自我进化 (Gao et al., 2025; Lopez-Paz and Ranzato, 2017) 的智能体，使其随着每条轨迹而变得更加胜任，而非从零开始。这推动了关于自我进化智能体记忆的一系列研究 (Ouyang et al., 2026; Wang et al., 2025b; Tang et al., 2025b; Zhang et al., 2025b)。早期工作将原始交互轨迹存储为扁平记录 (Wang et al., 2023a; Zheng et al., 2024; Zhao et al., 2024)，而后续方法转向将经验提炼为更紧凑、可迁移的单元，如工作流 (Wang et al., 2025b)、推理策略 (Ouyang et al., 2026) 和执行技能 (Xia et al., 2026)。尽管取得了这些进展，但经验记忆的设计仍存在不足。

¹Code is available at [\[link\]](#)

经验记忆系统仍是一个开放问题，可分解为三个基本问题：存储什么、如何结构化与检索、以及记忆如何演化。现有系统仅部分解决每个问题，留下显著差距，限制了其自我改进能力。在存储什么方面，早期系统保留原始轨迹或将其提炼为洞见与工作流（Zhao 等，2024；Wang 等，2023a，2025b），而近期工作将经验抽象为可复用技能（Ouyang 等，2026；Zheng 等，2025；Anthropic，2024）。然而，这些系统丢弃了子任务与技能之间的组合关系。由于任务本质上是组合性的，当这些关系未被保留时，检索无法利用表面不同任务间共享的程序结构。在如何结构化与检索方面，大多数方法采用扁平向量存储或简单二元图（Zhang 等，2025a；Chhikara 等，2025；Rasmussen 等，2025），仅通过嵌入相似度检索，无法表示技能与其反复成功的子任务模式之间的 $n \times n$ 元共现。在记忆如何演化方面，大多数系统无任何策展地积累知识（Wang 等，2023a；Zhao 等，2024），少数引入仅轻量操作如剪枝或去重（Fang 等，2026；Zheng 等，2025），独立应用于每个节点。这些差距共享一个共同的结构根源：轨迹不是单一知识单元，而是子任务、技能与结果之间的 n 元关联，扁平列表与成对边只能有损表示这种结构。超图（Zhou 等，2006；Tang 等，2025a）通过让每条超边同时连接两个以上节点，原生保留此类关联。我们提出 HYPERSKILL，将经验记忆组织为超图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ ，包含子任务节点 $\mathcal{V}_u$ 与技能节点 $\mathcal{V}_s$ ，每条超边将单条轨迹中的子任务与技能连同提炼的教训与效用分数分组。此设计直接改善三个差距：❶超边保留扁平存储丢弃的完整组合上下文；❷技能节点跨超边共享，使我们能通过候选技能在检索轨迹中出现的频率进行排序，这是仅靠语义相似度无法提供的结构信号；❸超图上的质量加权传播通过联合合并冗余技能

考虑节点效用与邻域拓扑。在三个智能体基准（xBench、GAIA 和 WebWalkerQA）上，针对两种不同的模型系列，HYPERSKILL 始终优于十种记忆基线。

2 相关工作

智能体系统中的记忆机制。大语言模型智能体日益配备外部记忆，以突破固定上下文窗口的限制（Huang et al., 2026），承担事实记忆（Gutiérrez et al., 2024; Rasmussen et al., 2025）、工作记忆（Packer et al., 2023）以及经验记忆等角色，其中经验记忆将智能体自身的轨迹提炼为可复用的知识，用于自我改进。本文聚焦于后者，并沿三个维度对既有工作进行分类，指出各维度中的不足。（一）存储内容：早期工作保留原始轨迹（Zheng et al., 2024; Wen et al., 2024），虽保留上下文但携带噪声；后续方法将轨迹抽象为洞见或工作流（Zhao et al., 2024; Ouyang et al., 2026; Wang et al., 2025b; Fang et al., 2026），以牺牲过程细节为代价获得泛化性；近期工作提炼可复用技能（Zheng et al., 2025; Wang et al., 2023a; Xia et al., 2026），但孤立地处理每个技能。尚无方法保留可复用的子任务分解，限制了跨情节的组合规划迁移。（二）记忆的结构与检索方式：多数系统采用扁平向量或 JSON 存储，并基于嵌入进行检索（Zhao et al., 2024; Ouyang et al., 2026; Wang et al., 2025b; Zheng et al., 2025）；少数采用图结构（Zhang et al., 2025a; Yang et al., 2026）以捕捉关系依赖，但成对边将 n 元共现分解为不连通的二元链接，丢弃了情节的联合组合上下文。（三）记忆的演化方式：多数系统无条件累积（Wang et al., 2023a; Zhao et al., 2024; Wang et al., 2025b）；少数增加轻量级逐项剪枝（Zheng et al., 2025）、整合（Zhang et al., 2025a）或去重（Tang et al., 2025b），但无一利用结构信号或同时考虑质量与图拓扑。更详细的讨论见附录 E。以及检索方式，以及记忆随时间的演化方式，指出各维度中的不足。（一）存储内容：早期工作存储整个情节的原始轨迹（Zheng et al., 2024; Wen et al., 2024），虽保留完整上下文但包含大量

初始噪声；后续方法将经验抽象为洞见与 workflow (Zhao 等, 2024; Ouyang 等, 2026; Wang 等, 2025b; Fang 等, 2026), 提升了泛化能力, 但丢弃了细粒度的过程细节；另一些方法从轨迹中蒸馏可复用技能 (Zheng 等, 2025; Wang 等, 2023a; Xia 等, 2026), 保留了简洁的知识单元, 却孤立地处理每个技能, 缺乏组合结构。现有系统未能维护可复用的子任务分解, 以捕捉解决任务所用的过程序列, 从而限制了智能体跨情节迁移组合规划知识的能力。(二) 记忆的结构与检索方式：多数系统将记忆存储为扁平向量数据库或 JSON 文件, 仅依赖语义相似度进行检索 (Zhao 等, 2024; Ouyang 等, 2026; Wang 等, 2025b; Zheng 等, 2025), 限制了所检索知识的相关性与全面性。少数工作探索图结构以捕捉关系依赖 (Zhang 等, 2025a; Yang 等, 2026; Xu 和 Ding, 2026), 支持结构化遍历, 但依赖成对边, 将 n 元共现分解为不连通的二元链接, 丢弃了情节的联合组合上下文。(三) 记忆的演化方式：多数系统无条件累积记忆 (Wang 等, 2023a; Zhao 等, 2024; Wang 等, 2025b); 少数添加轻量管理, 如剪枝 (Zheng 等, 2025)、整合 (Zhang 等, 2025a) 或去重 (Tang 等, 2025b), 但均作用于单个条目, 未利用结构信息, 缺乏由质量信号与图拓扑联合引导的维护。更详细的讨论见附录 E。

3 超技能

3.1 预备知识

智能体、环境与记忆。沿用先前工作 (Zhang 等, 2025b), 我们将基于大语言模型的智能体系统形式化为策略 π_θ , 在轨迹上与环境交互, 其中 d_i 为任务描述, o_i 与 a_i $\tau_i = (d_i, o_i, a_i, r_i)$ 分别为观测序列与动作序列, $r_i \in \{\text{成功}, \text{失败}\}$ 为结果。在每一步 t , 智能体选择 $a_t \sim \pi_\theta(\cdot | o_t, m_t)$, 其中 m_t 是从外部记忆模块 $\mathcal{M}$ 中检索到的记忆上下文。给定不断增长的历史 $\mathcal{H} = \{\tau_1, \dots, \tau_N\}$, $\mathcal{M}$ 支持检索 $m_t = g_{\mathcal{M}}(d, \mathcal{H})$ 与更新 $\mathcal{M} \leftarrow \text{UPDATE}(\mathcal{M}, \tau_i)$,

以最大化期望任务性能为目标：

$$\max_{\mathcal{M}} \mathbb{E}_{d \sim \mathcal{D}} [R(\pi_\theta(\cdot | d, g_{\mathcal{M}}(d, \mathcal{H})))] . \quad (1)$$

3.2 HYPERSKILL 记忆架构

超图 (Hypergraph) (Zhou 等人, 2006) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 通过允许每条超边 $e \in \mathcal{E} \subseteq 2^{\mathcal{V}}$ 连接任意节点子集 ($|e| \geq 2$) 来推广普通图, 捕获多个节点之间成对边无法表示的高阶 n 元关系。HYPERSKILL 利用这种结构在单一统一记忆中同时编码组合式任务分解和可复用的执行知识。[>] 节点。每个 $v_i \in \mathcal{V}$ 是从智能体经验中提炼出的离散、可复用的任务知识单元。节点集划分为 $\mathcal{V} = \mathcal{V}_u \cup \mathcal{V}_s$, 其中每个节点可以用一个元组表示：

$$v_i \triangleq (c_i, \ell_i, \gamma_i), \quad \ell_i \in \{u, s\}, \quad (2)$$

其中 c_i 为自然语言内容, ℓ_i 为类型标签, $\gamma_i \in [0, 1]$ 为效用分数, 这是一种动态测度, 用于跟踪经验成功率和步骤效率。具体而言, 对于在轨迹 τ_i 期间参与检索的所有节点和超边, 我们在成功时递增 $\sigma(\cdot)$, 并用步数 T_i 更新 $\bar{T}(\cdot)$, 然后重新计算：

$$\gamma(\cdot) = \beta \cdot \frac{\sigma(\cdot)}{\nu(\cdot)} + (1 - \beta) \cdot \left(1 - \frac{\bar{T}(\cdot) - T_{\min}}{T_{\max} - T_{\min}}\right), \quad (3)$$

其中 $\nu(\cdot)$ 为总检索次数, $\sigma(\cdot)$ 为成功次数, $\bar{T}(\cdot)$ 为运行平均步数, β 用于平衡成功率与步骤效率。两种节点类型反映了任务知识的互补方面：子任务节点编码任务的组合式 workflow 结构, 而技能节点编码可跨任务迁移的可复用执行策略。○ 子任务节点 $\mathcal{V}_u$ ：每个 $v_u \triangleq (c_u, u, \gamma_u)$ 捕获任务分解中的一个步骤。内容 c_u 编码步骤描述及其适用的任何前提条件。

○ 技能节点 $\mathcal{V}_s$ ：每个 $v_s \triangleq (c_s, s, \gamma_s)$ 捕获通过结果条件提示提取的可复用知识。内容 c_s 编码简洁的策略标签和描述, 涵盖做什么、何时应用以及为何有效（或失败）。成功的轨迹产生可遵循的正向策略；失败的轨迹产生需要避免的错误模式。

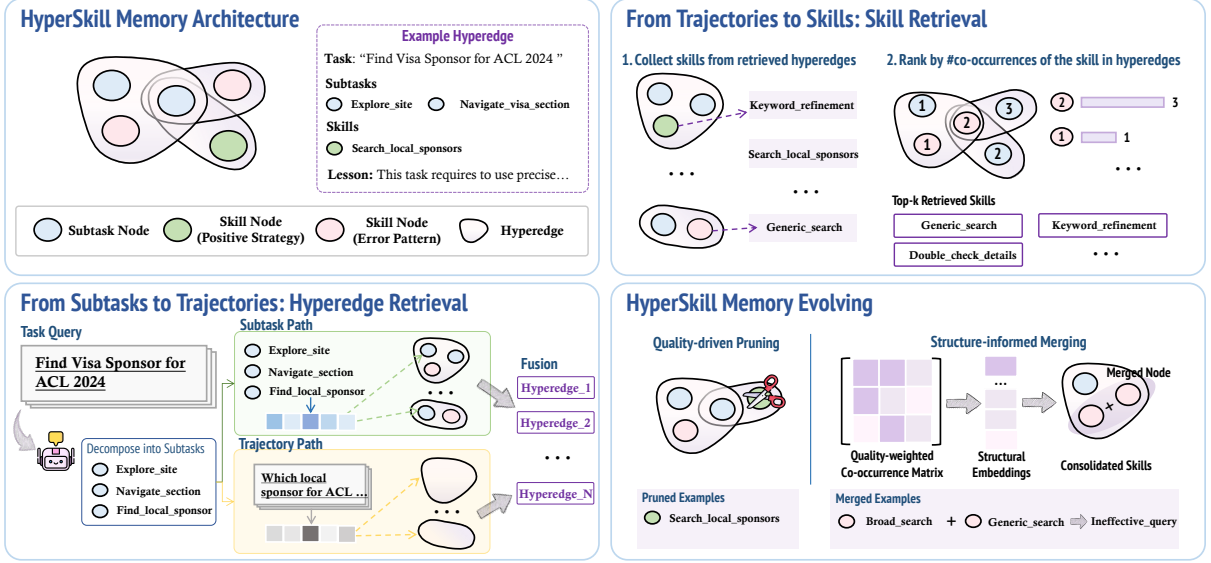


图 2: HYPERSKILL 概览。给定一个新任务，智能体将其分解为子任务，通过双路径超边检索获取相关轨迹，根据共现对技能进行排序，并利用检索到的上下文执行。任务完成后，新节点和一条超边被添加到记忆中，记忆会定期进行剪枝和合并。

[>] 超边。每条超边 $e_j \in \mathcal{E}$ 对应一条单一轨迹 τ_j 并定义为一个元组 $e_j \triangleq (V_j, d_j, \ell_j, \gamma_j)$ ，其中 $V_j = V_j^u \cup V_j^s$ ，与 $V_j^u \subseteq \mathcal{V}_u$ 子任务节点和 $V_j^s \subseteq \mathcal{V}_s$ 从提取的技能节点 τ_j ; d_j 是原始任务描述；以及 ℓ_j 是一条精炼的蒸馏经验，捕捉了轨迹所揭示的可迁移模式。在检索时，每条超边由嵌入表示 $\mathbf{h}_e = \phi(d_j \parallel \ell_j)$ ，该嵌入将任务描述与蒸馏经验拼接，使相似度匹配能够同时捕捉表面措辞与底层可迁移模式。

3.3 从子任务到轨迹：

超边检索

现有方法探索了多种形式的经验知识，从原始轨迹 (Zheng 等, 2024; Wen 等, 2024) 和提炼洞见 (Ouyang 等, 2026; Zhao 等, 2024) 到可复用技能 (Zheng 等, 2025; Wang 等, 2023a)，但大多数仅依赖完整任务描述作为唯一检索查询。这忽略了表面不同但共享相似程序性子任务模式的轨迹。为检索更全面的轨迹级上下文，HYPERSKILL 通过两条互补路径查询超图：一条将当前任务分解为子任务，并与存储的子任务节点匹配，从而找出共享程序结构的轨迹，无论任务级措辞如何；另一条直接将任务描述与超边表示匹配，

超边表示同时编码原始任务及其提炼经验。任务分解与子任务路径。收到新任务 d_q 后，智能体首先调用一次大语言模型生成细粒度分解 $\mathcal{P}_0 = \{d_1, \dots, d_m\}$ ，其中每个 d_i 描述一个子任务单元。该分解与超图中已存储的子任务结构一致，从而支持在子任务级别而非完整任务级别进行检索。我们使用定长文本编码器 ϕ 将完整分解编码为单个向量 $\mathbf{h}_{\mathcal{P}_0} = \phi(\mathcal{P}_0)$ ，并检索嵌入 $\mathbf{h}_v = \phi(c_v)$ 最相似的前 k_u 个子任务节点：

$$R_u = \text{top-}k_u \text{ sim}(\mathbf{h}_{\mathcal{P}_0}, \mathbf{h}_v)_{v \in \mathcal{V}_u}. \quad (4)$$

由于每个子任务节点关联一条或多条超边，匹配到的节点自然扩展为一组候选轨迹： $\mathcal{E} \mid V_e \cap R_u \neq \emptyset$ 。超边 e 的节点集。该路径在找出与当前任务共享程序子结构的轨迹方面尤为有效，即使其任务级描述差异显著。

轨迹路径与融合。 仅依赖子任务路径可能会遗漏那些整体任务描述高度相关、但单个子任务节点未进入前 k_u 匹配的轨迹。互补的轨迹级路径通过将完整任务描述编码为 $\mathbf{h}_{d_q} = \phi(d_q)$ 并直接对超边进行排序来解决这一问题

by similarity to the query:

$$\mathcal{E}_{\text{traj}} = \text{top-}k_e \text{ sim}(\mathbf{h}_{d_q}, \mathbf{h}_e), \quad (5)$$

其中 $\mathbf{h}_e = \phi(d_e \parallel \ell_e)$ 将原始任务描述与提炼出的经验拼接，使排序同时捕捉表面措辞和底层可迁移模式。两条路径的候选结果合并为统一的轨迹集合：

$$\mathcal{E}^* = \mathcal{E}_{\text{sub}} \cup \mathcal{E}_{\text{traj}}. \quad (6)$$

两条路径互为补充：子任务路径检索具有共享程序性子结构的轨迹，而轨迹路径检索具有相似高层任务语义的轨迹。融合集合中的提炼经验 $\mathcal{L}_q = \{\ell_e \mid e \in \mathcal{E}^*\}$ 在执行期间作为轨迹级上下文提供给智能体。

3.4 从轨迹到技能：技能检索

给定融合轨迹集合 $\mathcal{E}^*$ ，下一步是选择哪些技能作为执行指导呈现。 $\mathcal{E}^*$ 中的每条超边包含从该轨迹提取的技能节点，因此候选池为它们的并集：

$$R_s = \bigcup_{e \in \mathcal{E}^*} V_e^s \subseteq \mathcal{V}_s, \quad (7)$$

其中 V_e^s 表示超边 e 的技能节点。扁平记忆系统仅根据与 d_q 的嵌入相似度对这些候选进行排序。然而，语义相似度仅衡量技能描述是否与当前任务相似；它无法捕捉该技能是否已在与当前任务相关的轨迹中实际应用。

共现排序。 HYPERSKILL 利用技能节点与子任务节点不同、可跨轨迹去重复用的特性。一个技能出现在许多检索到的轨迹中，说明它已在任务相关情境中被反复应用，因此更可能有用。我们用一个简单的共现计数将其形式化：对每个候选 $v \in R_s$ ，统计有多少检索到的超边包含 v ：

$$\kappa(v) = |\{e \in \mathcal{E}^* \mid v \in V_e\}|. \quad (8)$$

按 κ 排名前 k_s 的技能被选为执行指导，平时时按嵌入相似度打破。

to d_q :

$$\mathcal{S}_q = \text{top-}k_s \kappa(v). \quad (9)$$

Execution and memory update

检索到的上下文 $(\mathcal{S}_q, \mathcal{L}_q)$ 每个任务组装一次，并在整个执行过程中保持不变，实例化记忆上下文 $m_t = (\mathcal{S}_q, \mathcal{L}_q)$ 。在每一步 t ，智能体根据当前观察 o_t 、检索到的技能 $\mathcal{S}_q$ 、提炼的经验教训 $\mathcal{L}_q$ 以及最近交互历史的滑动窗口来选择动作：

$$a_t \sim \pi_\theta(\cdot \mid o_t, \mathcal{S}_q, \mathcal{L}_q, \text{hist}_{t-w:t-1}). \quad (10)$$

任务完成后，根据结果 r_i 和步数 T_i ，以结果为条件的 LLM 调用从完整轨迹中提取技能节点 S_i^{new} 和提炼的经验教训 ℓ_{e_i} 。子任务节点 U_i^{new} 直接取自 $\mathcal{P}_0$ 。在将每个新提取的技能节点插入 $\mathcal{V}_s$ 之前，我们计算它与所有现有技能节点的嵌入相似度；如果相似度超过阈值 δ_{dedup} ，新节点将合并到其最近的现有邻居中，而不是作为重复项添加，从而保持技能词汇表的紧凑性和可重用性。然后在 $\mathcal{E}$ 上添加一个覆盖 $U_i^{\text{new}} \cup S_i^{\text{new}}$ 的新超边，并附带经验教训 ℓ_{e_i} 和任务描述 d_q ；其嵌入为 $\phi(d_q \parallel \ell_{e_i})$ ，与公式 5 一致。效用分数 $\gamma(\cdot)$ 通过公式 3 对所有参与检索的元素重新计算。

3.5 HYPERSKILL 记忆演化

随着 HYPERSKILL 积累任务， $\mathcal{G}$ 面临过时的风险：节点可能在语义上变得冗余，而反复检索但质量低下的知识会主动误导未来的决策。为了保持记忆紧凑、非冗余且高质量，HYPERSKILL 每 N_{maint} 个任务定期通过两种操作来优化 $\mathcal{G}$ ：质量驱动的剪枝和结构引导的合并。

质量驱动的剪枝。 那些被检索次数足够多但始终未能改善智能体结果的节点会被明确地从 $\mathcal{G}$ 中移除：

$$\mathcal{V} \leftarrow \mathcal{V} \setminus \{v \in \mathcal{V} \mid \nu(v) \geq N_{\text{min}} \wedge \gamma(v) < \tau_{\text{prune}}\}, \quad (11)$$

其中 $\nu(v)$ 是 v 被检索的总次数， $\gamma(v)$ 是其综合质量分数（公式 3），反映了在这些检索过程中积累的经验成功率和步骤效率。当节点被删除时，它会从所有超边成员关系中移除。

结构信息引导的合并。 表现出强质量加权共现且编码语义相似执行知识的技能节点是

表 1: 三个基准上的主要结果。我们报告成功率 (SR %)、平均步数 (#Steps) 和每个任务的平均工具调用次数 (#Calls)。最佳; 次佳按骨干网络。↑/↓: 与无记忆相比的变化。对于 #Steps 和 #Calls, 越低越好。

Backbone	Method	xBench			GAIA			WebWalkerQA		
		SR	#Steps	#Calls	SR	#Steps	#Calls	SR	#Steps	#Calls
 Qwen3-30B-A3B	No Memory	46.00 _{0.00}	4.87 _{0.00}	5.05 _{0.00}	32.12 _{0.00}	8.23 _{0.00}	4.75 _{0.00}	39.41 _{0.00}	4.07 _{0.00}	3.87 _{0.00}
	ReasoningBank	43.00 _{0.00}	4.47 _{0.40}	4.72 _{0.33}	29.70 _{0.42}	7.44 _{0.79}	4.42 _{0.33}	44.71 _{0.30}	3.64 _{0.43}	3.72 _{0.15}
	ExpeL	45.00 _{0.00}	5.14 _{0.27}	6.06 _{0.01}	33.33 _{0.21}	7.38 _{0.85}	4.15 _{0.60}	47.06 _{0.65}	4.04 _{0.03}	3.94 _{0.07}
	AWM	44.00 _{0.00}	4.86 _{0.01}	5.49 _{0.44}	32.73 _{0.61}	7.07 _{0.16}	3.92 _{0.83}	41.76 _{0.35}	3.84 _{0.23}	3.72 _{0.15}
	Generative	49.00 _{0.00}	4.46 _{0.41}	4.97 _{0.08}	34.55 _{0.43}	7.43 _{0.80}	3.97 _{0.78}	41.18 _{0.77}	3.82 _{0.25}	3.87 _{0.00}
	Voyager	50.00 _{0.00}	4.77 _{0.10}	5.05 _{0.00}	34.55 _{0.43}	7.47 _{0.76}	4.16 _{0.59}	46.47 _{0.66}	3.84 _{0.23}	3.82 _{0.05}
	DILU	52.00 _{0.00}	5.01 _{0.14}	5.45 _{0.40}	32.12 _{0.00}	7.14 _{0.09}	4.17 _{0.58}	44.12 _{0.71}	3.87 _{0.20}	3.95 _{0.08}
	Cheatsheet	37.00 _{0.00}	4.03 _{0.84}	4.84 _{0.21}	27.88 _{0.24}	6.84 _{0.39}	5.28 _{0.53}	39.41 _{0.00}	3.62 _{0.45}	4.06 _{0.19}
	MemP	45.00 _{0.00}	4.42 _{0.45}	4.24 _{0.81}	32.73 _{0.61}	6.97 _{0.26}	4.42 _{0.33}	45.88 _{0.47}	3.72 _{0.35}	3.66 _{0.21}
	MemEvolve	39.00 _{0.00}	4.64 _{0.23}	4.72 _{0.33}	28.48 _{0.64}	5.08 _{0.15}	3.67 _{0.08}	37.06 _{0.35}	3.61 _{0.46}	3.32 _{0.55}
	PlugMem	42.00 _{0.00}	5.07 _{0.20}	5.18 _{0.13}	34.55 _{0.43}	7.02 _{0.21}	4.18 _{0.57}	35.88 _{0.53}	3.96 _{0.11}	3.83 _{0.04}
 GPT-4o	HYPERSKILL	52.00 _{0.00}	4.52 _{0.35}	4.88 _{0.17}	36.97 _{0.85}	7.25 _{0.98}	5.48 _{0.73}	50.59 _{0.18}	4.04 _{0.03}	4.06 _{0.19}
	No Memory	54.00 _{0.00}	6.29 _{0.00}	6.33 _{0.00}	32.73 _{0.00}	6.59 _{0.00}	8.23 _{0.00}	46.47 _{0.00}	5.82 _{0.00}	7.04 _{0.00}
	ReasoningBank	57.00 _{0.00}	8.22 _{0.93}	12.27 _{0.94}	33.33 _{0.60}	7.21 _{0.62}	10.17 _{0.94}	47.65 _{0.18}	5.59 _{0.23}	6.45 _{0.59}
	ExpeL	54.00 _{0.00}	5.27 _{0.02}	6.11 _{0.22}	35.76 _{0.03}	6.63 _{0.04}	9.48 _{0.25}	47.06 _{0.59}	5.58 _{0.24}	6.32 _{0.72}
	AWM	52.00 _{0.00}	6.47 _{0.18}	8.12 _{0.79}	35.15 _{0.42}	7.13 _{0.54}	9.08 _{0.85}	47.65 _{0.18}	5.77 _{0.05}	6.49 _{0.55}
	Generative	56.00 _{0.00}	7.65 _{0.36}	11.34 _{0.01}	36.36 _{0.63}	6.39 _{0.20}	7.68 _{0.55}	51.18 _{0.71}	5.77 _{0.05}	6.33 _{0.71}
	Voyager	47.00 _{0.00}	7.99 _{0.70}	11.27 _{0.94}	31.52 _{0.21}	6.52 _{0.07}	9.15 _{0.92}	48.82 _{0.35}	6.07 _{0.25}	6.93 _{0.11}
	DILU	53.00 _{0.00}	7.16 _{0.87}	9.98 _{0.65}	33.33 _{0.60}	5.34 _{0.25}	5.63 _{0.60}	48.24 _{0.77}	6.05 _{0.23}	6.97 _{0.07}
	Cheatsheet	55.00 _{0.00}	7.56 _{0.27}	10.92 _{0.59}	35.15 _{0.42}	6.27 _{0.32}	8.36 _{0.13}	47.06 _{0.59}	6.02 _{0.20}	7.25 _{0.21}
	MemP	48.00 _{0.00}	7.59 _{0.30}	10.76 _{0.43}	33.33 _{0.60}	7.04 _{0.45}	8.64 _{0.41}	50.59 _{0.12}	5.66 _{0.16}	6.03 _{0.01}
	MemEvolve	47.00 _{0.00}	6.68 _{0.39}	9.06 _{0.73}	36.36 _{0.63}	6.97 _{0.38}	7.29 _{0.94}	48.24 _{0.77}	6.09 _{0.27}	7.34 _{0.30}
	PlugMem	59.00 _{0.00}	5.94 _{0.35}	6.22 _{0.11}	41.94 _{0.21}	5.73 _{0.86}	5.11 _{0.12}	47.06 _{0.59}	5.96 _{0.14}	6.49 _{0.55}
	HYPERSKILL	62.00 _{0.00}	5.27 _{0.02}	6.52 _{0.19}	44.24 _{0.51}	6.01 _{0.58}	7.35 _{0.88}	51.18 _{0.71}	5.93 _{0.11}	7.04 _{0.00}

合并为单一高阶技能的候选。子任务节点 v_u 被排除, 因为合并规划步骤会破坏任务分解的组合结构。为了在 V_s 内识别合并候选, 我们计算质量加权结构嵌入。我们首先构建共现矩阵

$$\mathbf{W} \in \mathbb{R}^{|V_s| \times |V_s|};$$

$$W_{ij} = \min(\gamma(v_i), \gamma(v_j)) \cdot \sum_{e \in \mathcal{E}} \frac{1}{|V_e|}, \quad (12)$$

其中 $1/|V_e|$ 根据同质性假设 (Tang 等, 2025a) 按超边大小进行归一化——共享小而紧密超边的节点比共享大超边的节点更可能编码相似知识——而 $\min(\gamma(v_i), \gamma(v_j))$ 确保只有当两个节点都有用时才出现高边权。然后通过 L 步对称传播获得结构嵌入, 遵循 (Tang 等, 2025a) :

$$\begin{aligned} \tilde{\mathbf{Z}} &= \mathbf{S} \mathbf{Z}, \\ \mathbf{S} &= (1 - \alpha)^L \hat{\mathbf{W}}^L + \alpha \sum_{l=0}^{L-1} (1 - \alpha)^l \hat{\mathbf{W}}^l, \quad (13) \\ \hat{\mathbf{W}} &= \hat{\mathbf{D}}^{-1/2} (\mathbf{W} + \mathbf{I}) \hat{\mathbf{D}}^{-1/2}, \end{aligned}$$

其中 $\mathbf{Z} \in \mathbb{R}^{|V_s| \times d}$ 堆叠普通技能节点嵌入, $\hat{\mathbf{D}}$ 是 $(\mathbf{W} + \mathbf{I})$, $\alpha \in (0, 1)$ 平衡个体节点身份与邻域上下文, L 控制传播深度。由于节点效用已编码在 $\mathbf{W}$ 中, 传播后的嵌入 $\tilde{\mathbf{Z}}$ 共同反映结构共现和技能质量, 并且一对 (v_a, v_b) 仅通过相似性被选为合并候选:

$$\text{sim}(\tilde{\mathbf{z}}_a, \tilde{\mathbf{z}}_b) \geq \delta_{\text{merge}}. \quad (14)$$

对于每个选定的对, 智能体根据 v_a, v_b 的内容及其共享轨迹片段进行推理, 提出一个合并节点 v_{new} , 将其知识整合为高阶技能。所有超边成员关系重新分配给 v_{new} , 保持 $\mathcal{G}$ 的结构完整性。

4 实验

4.1 实验设置

数据集与基准 涵盖不同领域与能力要求的多样化基准上对超技能 (HY-PERSKILL) 进行评估: (一) x 基准 (xBench) (Chen 等人, 2025), 该基准用于评估智能体规划、工具使用及多步骤能力。

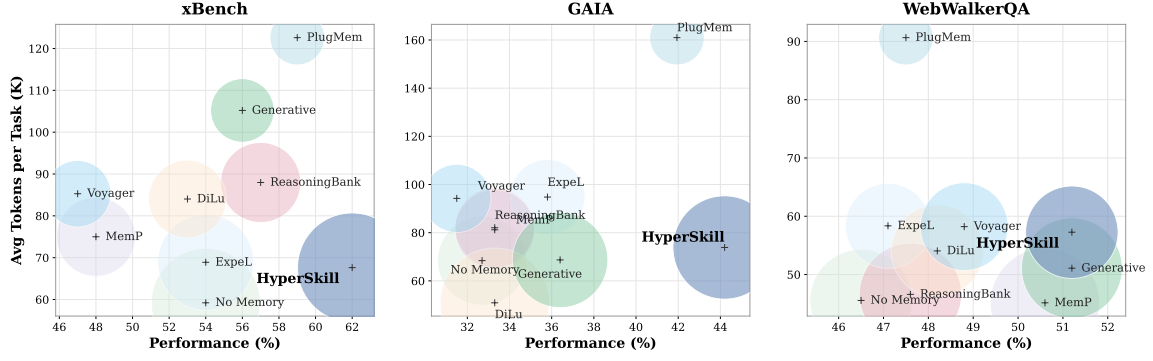


图 3: 三个基准上性能与每任务平均令牌数 (K) 的关系。气泡大小反映性能与令牌数之比。

在广泛的现实世界任务中进行推理；

(二) 通用人工智能助手基准 (GAIA) (米亚隆等人, 2023 年), 这是一组需要多跳推理、多模态处理、网页浏览和工具使用能力的真实世界问题; 以及 (三) 网页行者问答

(WebWalkerQA) (吴等人, 2025 年 a), 这是一个测试跨多种真实世界查询和网页的复杂多轮网页导航的基准。这些基准共同涵盖了工具增强推理、开放式网页交互和组合规划。更多细节见附录 A。

基线。 本文将 HYPERSKILL 与十种代表性记忆系统进行比较, 这些系统涵盖三个类别。轨迹级方法存储原始或轻度处理的轨迹: Voyager (Wang 等, 2023a)、DILU (Wen 等, 2024)、ExpeL (Zhao 等, 2024) 和 Generative (Shang 等, 2025)。工作流或洞见级方法提取并管理可复用的知识单元: AWM (Wang 等, 2025b)、ReasoningBank (Ouyang 等, 2026)、MemP (Fang 等, 2026)、Cheatsheet (Suzgun 等, 2026) 和 MemEvolve (Zhang 等, 2025b)。基于图的方法利用关系结构组织记忆: PlugMem (Yang 等, 2026)。本文还纳入一个无记忆基线, 该基线不使用任何外部记忆模块。所有基线均采用相同的智能体骨干和工具访问权限, 以确保公平比较。更多细节见附录 A。

实现细节。 所有实验均基于两种骨干模型开展: GPT-4o (Hurst 等人, 2024) 与 Qwen3-30B-A3B (Yang 等人, 2025), 涵盖闭源与开源两种设置。对于 HYPERSKILL, 本文采用 all-MiniLM-L6-v2 作为共享编码器 $\phi(\cdot)$, 用于所有节点与超边的嵌入。单一检索预算同时约束于任务节点检索、超边检索及最终技能排序。

维护每 $[0.1 \times N]$ 个回合触发一次, 剪枝阈值为 $\tau_{\text{prune}}=0.2$, 最小检索次数为 $N_{\text{min}}=3$ 。效用得分混合系数为 $\beta=0.7$ 。所有超参数与实现细节见附录 A。

4.2 主要结果

表 1 报告了在 xBench、GAIA 和 WebWalkerQA 三个基准上, 两种骨干模型下的成功率 (SR)、平均步数和平均工具调用次数。HYPERSKILL 在几乎所有基准-骨干组合上都达到了最高或接近最高的成功率。我们强调三点观察。❶跨基准和骨干的一致提升。在 GPT-4o 下, HYPERSKILL 优于所有基线

on SR by +3.00 在 xBench 上, +2.30 在 GAIA 上, 以及 +0.59 在 WebWalkerQA 上相对于次优方法。在 Qwen3-30B-A3B 下, 该趋势依然成立, 分别带来 +6.00, +4.85、+11.18 的增益, 优于无记忆方法, 证实改进源于记忆框架而非骨干网络特定行为。❷提升的成功率

且不增加执行成本。 关于记忆增强智能体 (Memory-Augmented Agent) 的一个常见顾虑是, 更丰富的检索上下文可能增加每步开销。HYPERSKILL 在很大程度上避免了这一问题: 在 xBench 上使用 GPT-4o 时, 它取得了最高的成功率 (SR), 同时以最少的步骤数 (5.27) 与 ExpeL 持平, 并保持了有竞争力的工具调用次数 (6.52)。相比之下, ReasoningBank 和 Voyager 等方法在同一基准上需要多达 8.22 步和 12.27 次调用, 却取得了更低的成功率, 这表明它们检索到的上下文引入了开销而没有带来成比例的收益。❸朴素的记忆累积

可能造成损害。 若干基线方法在至少一种设置下表现不如无记忆 (No Memory): Cheatsheet 在 xBench (Qwen3-30B-A3B) 上下降了 -9.00, Voyager 在 xBench (GPT-4o) 上下降了 -7.00。在没有原则性

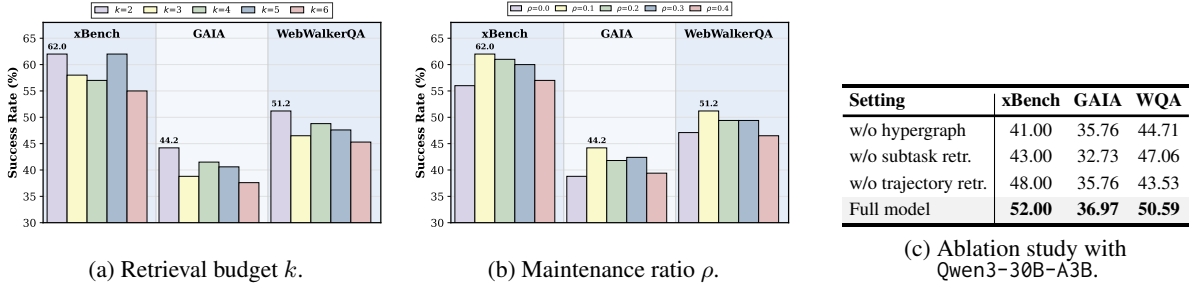


图 4: 敏感性分析与消融实验。(a, b) 在 xBench、GAIA 和 WebWalkerQA 上, 当检索预算 k 和维护比率 ρ 变化时, GPT-4o 的成功率 (%)。 (c) HYPERSKILL 的轨迹检索、子任务检索和超图结构的消融。

维护的情况下, 累积的条目会引入噪声, 主动损害智能体的性能。这凸显了 HYPERSKILL 的结构感知演化机制的重要性, 该机制剪除低效用节点并合并冗余技能, 以保持记忆的紧凑性和高质量。

4.3 分析与消融实验

成本分析。图 3 绘制了每个任务的平均令牌数与性能的关系, 气泡大小与性能-令牌比成正比。HYPERSKILL 在所有三个基准上都位于右下角区域, 以适中的令牌成本 (在 xBench、GAIA 和 WebWalkerQA 上分别为 67K、72K 和 57K) 取得了最高的性能。Generative 和 Voyager 等方法消耗了超过 100K 令牌, 但性能远低于 HYPERSKILL。PlugMem 尽管拥有以知识为中心的内存图, 但其令牌成本远高于 HYPERSKILL, 而性能却更低。差距源于记忆构建: PlugMem 需要额外的 LLM 调用来提取状态、子目标和奖励标注。

敏感性分析。本文分析了 HYPERSKILL 对两个关键超参数的敏感性: (i) 检索预算 k , 控制超边和技能检索过程中浮现的候选数量; (ii) 维护比例 ρ , 控制每个维护周期中剪枝的低效用条目比例。图 4 (a-b) 揭示了两个一致的模式。首先, 性能偏好较小的检索预算: $k=2$ 在全部三个基准上取得最佳结果, 性能随 k 增大而下降。这表明紧凑的检索集合能最小化低排名候选带来的噪声, 浮现过多条目会稀释最相关技能的信号。其次, 维护比例在 $\rho=0.1$ 处达到峰值, $\rho=0.2$ 与 $\rho=0.3$ 表现相近, 表明

轻度剪枝足以保持记忆的有效性。完全禁用剪枝 ($\rho=0$) 会让低效用条目累积并降低检索质量, 而过度激进的剪枝 ($\rho=0.4$) 会移除仍有用的技能。

消融实验。本文对两条检索路径及超图结构进行消融, 结果见图 4 (c)。无轨迹检索移除将完整任务描述与超边嵌入匹配的路径, 仅依赖将计划分解为子任务节点并通过其关联超边扩展。在 WQA 上性能显著下降 ($50.59 \rightarrow 43.53$), 表明轨迹级匹配捕获了仅靠子任务扩展会遗漏的任务全局模式。无子任务检索移除细粒度路径, 仅从任务匹配的前 k 条超边收集技能, 不进行计划分解或子任务级扩展。这对 xBench 影响最大, 并使 GAIA 降至 32.73, 表明子任务分解对于在复杂多步任务上检索相关技能至关重要。无超图将整个双路径流程替换为对存储技能的扁平文本嵌入检索, 移除所有超图结构。这在全部三个基准上产生最低结果, 证实超图的 n 元共现关系相比标准向量存储基线提供了有意义的增益。结合两条检索路径的完整模型在所有基准上取得最佳结果, 验证了它们的互补性。

4.4 超图分析

为说明 HYPERSKILL 的超图如何组织经验知识, 图 5 可视化了来自 xBench 的六条轨迹的代表性子图。子任务节点大多与特定轨迹相关, 而技能节点则跨超边共享: 跨源验证与定向数据库检索各自出现在多条轨迹中,

- Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, Shanglin Wu, Ruiyao Xu, Liangwei Yang, Rui Yang, Woosung Yang, Chin-Yuan Yeh, Hanrong Zhang, Haozhen Zhang, Siqi Zhu, and 41 others. 2026. A survey of agent memory in the second half: Towards self-evolving and long-horizon agents. *TMLR*.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.
- Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. 2026. Magma: A multi-graph based agentic memory architecture for ai agents. In *ACL*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for "mind" exploration of large language model society. *NeurIPS*.
- Bang Liu, Xinfeng Li, Jiayi Zhang, Jinlin Wang, Tanjin He, Sirui Hong, Hongzhang Liu, Shaokun Zhang, Kaitao Song, Kunlun Zhu, and 1 others. 2025a. Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. *arXiv preprint arXiv:2504.01990*.
- Yitao Liu, Chenglei Si, Karthik R Narasimhan, and Shunyu Yao. 2025b. [Contextual experience replay for self-improvement of language agents](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14179–14198, Vienna, Austria. Association for Computational Linguistics.
- David Lopez-Paz and Marc’Aurelio Ranzato. 2017. Gradient episodic memory for continual learning. In *NeurIPS*.
- Haoran Luo, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, and 1 others. 2025. Hypergraphrag: Retrieval-augmented generation via hypergraph-structured knowledge representation. In *NeurIPS*.
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. Gaia: a benchmark for general ai assistants. In *ICLR*.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. Reasoning-bank: Scaling agent self-evolving with reasoning memory. In *ICLR*.
- Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: towards llms as operating systems. *arXiv preprint arXiv:2305.00000*.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.
- Shuo Ren, Can Xie, Pu Jian, Zhenjiang Ren, Chunlin Leng, and Jiajun Zhang. 2025. Towards scientific intelligence: A survey of llm-based scientific agents. *arXiv preprint arXiv:2503.24047*.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. 2025. Agentsquare: Automatic LLM agent search in modular design space. In *ICLR*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *NeurIPS*.
- Haotian Sun, Yuchen Zhuang, Ling kai Kong, Bo Dai, and Chao Zhang. 2023. Adaplanner: Adaptive planning from feedback with language models. In *NeurIPS*.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. [Dynamic cheat-sheet: Test-time learning with adaptive memory](#). In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, Rabat, Morocco. Association for Computational Linguistics.
- Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314*.
- Bohan Tang, Zexi Liu, Keyue Jiang, Siheng Chen, and Xiaowen Dong. 2025a. Training-free message passing for learning on hypergraphs. In *ICLR*.
- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, and 1 others. 2025b. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023a. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv: Arxiv-2305.16291*.

- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023b. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *ACL*.
- Zhenhailong Wang, Haiyang Xu, Junyang Wang, Xi Zhang, Ming Yan, Ji Zhang, Fei Huang, and Heng Ji. 2025a. Mobile-agent-e: Self-evolving mobile assistant for complex tasks. *arXiv preprint arXiv:2501.11733*.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025b. Agent workflow memory. In *ICML*.
- Tianxin Wei, Ting-Wei Li, Zhining Liu, Xuying Ning, Ze Yang, Jiaru Zou, Zhichen Zeng, Ruizhong Qiu, Xiao Lin, Dongqi Fu, and 1 others. 2026. Agentic reasoning for large language models. *arXiv preprint arXiv:2601.12538*.
- Licheng Wen, Daocheng Fu, Xin Li, Xinyu Cai, Tao Ma, Pinlong Cai, Min Dou, Botian Shi, Liang He, and Yu Qiao. 2024. Dilu: A knowledge-driven approach to autonomous driving with large language models. In *ICLR*.
- Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and Fei Huang. 2025a. [WebWalker: Benchmarking LLMs in web traversal](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10290–10305, Vienna, Austria. Association for Computational Linguistics.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, and 1 others. 2024. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *COLM*.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xueming Yang, Yufan Shen, Yuxin Wang, and 1 others. 2025b. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*.
- Zhaofen Wu, Hanrong Zhang, Fulin Lin, Wujiang Xu, Xinran Xu, Yankai Chen, Henry Peng Zou, Shaowen Chen, Weizhi Zhang, Xue Liu, and 1 others. 2026. Gam: Hierarchical graph-based agentic memory for llm agents. *arXiv preprint arXiv:2604.12285*.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, and 1 others. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, and 1 others. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, and 1 others. 2024. Oworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS*.
- Ruiyao Xu and Kaize Ding. 2026. GNN-as-judge: Unleashing the power of LLMs for graph learning with GNN feedback. In *ICLR*.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026. A-mem: Agentic memory for LLM agents. In *NeurIPS*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*.
- Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and Chengxiang Zhai. 2026. Plugmem: A task-agnostic plugin memory module for llm agents. *arXiv preprint arXiv:2603.03296*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *ICLR*.
- Juwei Yue, Chuanrui Hu, Jiawei Sheng, Zuyi Zhou, Wenyuan Zhang, Tingwen Liu, Li Guo, and Yafeng Deng. 2026. Hypermem: Hypergraph memory for long-term conversations. In *ACL*.
- Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. 2025a. G-memory: Tracing hierarchical memory for multi-agent systems. In *NeurIPS*.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. 2025b. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *AAAI*.

Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and 1 others. 2025. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*.

Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2024. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *ICLR*.

Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. 2006. Learning with hypergraphs: Clustering, classification, and embedding. In *NeurIPS*.

附录概览 • 附录 A：数据集与实现细节

- 附录 B：更多分析
- 附录 C：更广泛影响
- 附录 D：大语言模型使用披露
- 附录 E：扩展相关工作
- 附录 F：伦理声明
- 附录 G：补充案例研究
- 附录 H：算法
- 附录 I：提示词

A 数据集与实现细节

A.1 数据集详情

本文遵循 MemEvolve (Zhang 等, 2025b) 的评估协议, 并采用相同的三个基准。GAIA (Mialon 等, 2023) 包含 165 个任务, 分为三个难度级别: 一级 (53 个任务)、二级 (86 个任务) 和三级 (26 个任务), 要求多跳推理、网页浏览和工具使用。**WebWalkerQA** (Wu 等, 2025a) 涵盖复杂的多轮网页导航, 包含四个领域的 680 个查询和超过 1,373 个网页。本文使用与 (Zhang 等, 2025b) 相同的 170 个查询子集。**xBench** (Chen 等, 2025) 包含 100 个任务, 评估在多样化真实场景中的智能体规划、工具使用和多步推理。

A.2 实现细节

本文基于 MemEvolve (Zhang 等, 2025b) 代码库构建, 并在所有方法中采用相同的智能体架构以确保公平比较。对于所有基线, 记忆检索数量设置为 3, 文本嵌入使用 all-MiniLM-L6-v2 计算, 并以余弦相似度作为检索度量。

大语言模型后端。对于 GPT-4o, 我们使用 OpenAI 应用程序编程接口, 温度参数为 0.7, 并设置检索预算为 $(k_u, k_e, k_s) = (2, 2, 2)$ 。对于 Qwen3-30B-A3B, 我们通过 vLLM (Kwon 等人, 2023) 在 NVIDIA A100 80 GB 图形处理器上本地部署模型, 温度参数为 0.7, 检索预算为 $(k_u, k_e, k_s) =$ 。近期交互历史的滑动窗口设置为 $w = 3$ 步。

记忆更新。技能去重阈值为 $\delta_{\text{dedup}} = 0.9$ ，未测试节点获得中性先验 $\gamma = 0.5$ 。效用分数混合系数

$$\beta = 0.7 \text{ (Eq. 3).}$$

维护。维护每 $\lceil 0.1 \times N \rceil$ 个任务触发一次，其中 N 是基准中的任务总数，剪枝阈值为 $\tau_{\text{prune}} = 0.2$ ，最小检索次数为 $N_{\text{min}} =$ 。合并候选阈值为 $\delta_{\text{merge}} = 0.85$ 。

可复现性。任务按固定的随机顺序（种子 42）依次处理，所有方法共享该顺序。

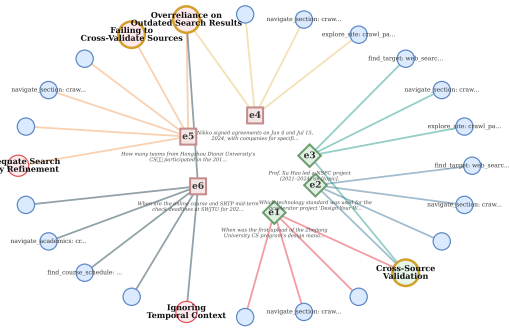


图 6: WebWalkerQA 上的超图子图可视化。轨迹超边（3 条成功，2 条失败）连接技能节点（○ 成功，○ 失败）和子任务节点（●）。

B 更多分析

B.1 自进化分析

为检验 HYPERSKILL 的超图记忆是否确实随经验而改进，我们绘制了三个基准上累计成功率随回合索引变化的曲线。

图 7 揭示了三个关键发现。首先，HYPERSKILL 是唯一表现出持续上升趋势的方法：在 GAIA 上，其累计准确率从早期回合到最终回合稳步攀升，证实超图记忆随时间积累了可迁移的技能。其次，最终成功率相当的基线方法（如 xBench 上的 DiLu）在早期达到峰值后停滞或中期下降，随后才恢复，表明其记忆并未跨任务复合。第三，自进化效应在任务多样性更高的基准（GAIA、WebWalkerQA）上比在 xBench 上更为显著，表明 HYPERSKILL 在超图能够捕捉多样化的跨任务技能共现模式时获益最大。

能够捕捉多样化的跨任务技能共现模式。

B.2 延迟分析

除成功率外，实际部署还要求可接受的端到端延迟。图 8 报告了两种骨干模型下三个基准的平均每任务执行时间。在 Qwen3-30B-A3B 下，HYPERSKILL 相对于最快基线（无记忆）增加了适度开销：xBench 上 42.6 秒对 26.5 秒，GAIA 上 33.6 秒对 33.6 秒（与 DILU 持平），WebWalkerQA 上 31.3 秒对 23.9 秒。在 GPT-4o 下，模式类似：HYPERSKILL 运行时间为 48.3 秒、47.6 秒和 55.3 秒，而最快基线分别达到 29.7 秒、32.5 秒和 41.9 秒。两种设置下，开销主要源于任务分解和技能提取所需的额外大语言模型调用，而非超图操作本身。结合图 3 中的令牌效率结果，这些结果证实 HYPERSKILL 以实用的延迟成本实现了最佳性能。

B.3 内存增长分析

图 9 展示了超图在连续任务中的演化过程。子任务节点大致呈线性增长，因为每个任务都会引入新的分解；而两类技能节点均呈亚线性增长：插入时去重可防止近似重复的技能进入图中，周期性的结构信息合并则整合冗余条目，二者共同使技能数量在所有基准和骨干模型上均远低于超边数量。

B.4 鲁棒性分析

实际应用中，每个任务之后可能无法获得真实标签，但遵循先前工作（Fang 等，2026；Zhang 等，2025b），HYPERSKILL 利用结果信号为提取的技能分配成功或失败标签。我们评估了一种自判断变体，其中智能体在无法访问标准答案的情况下评估自身轨迹结果，并将其与主实验中使用的真实判断设置进行比较。表 2 报告了 GPT-4o 上的结果。自判断变体在三个基准上均保留了真实判断性能的 87.7% 至 96.8%，其中在 xBench 上差距最小（-2.00 SR），在 GAIA 上差距最大（-5.37 SR）。适度的性能下降表明 HYPERSKILL 对噪声结果信号具有鲁棒性：对多个检索到的超边进行共现排序稀释了

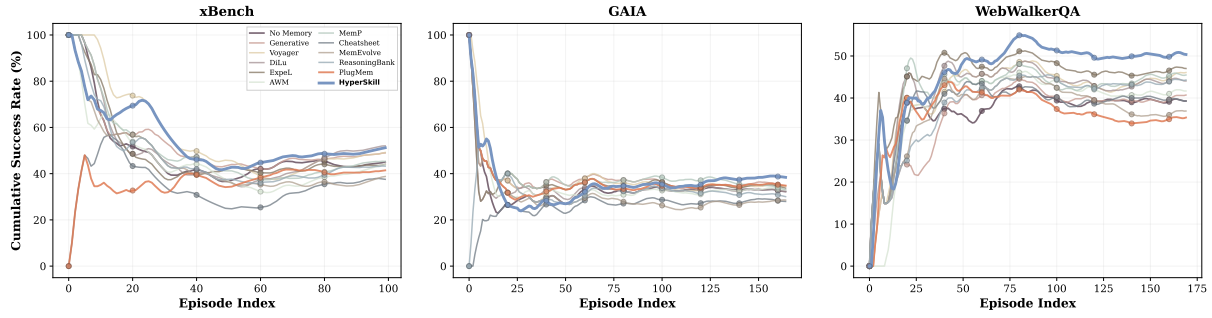


图 7: Qwen3-30B-A3B 上按回合索引的累积成功率 (%)。HYPERSKILL 在所有基准上均呈现上升趋势。

表 2: GPT-4o 上真实判断与自判断的比较。保留率=自判断 SR/真实判断 SR $\times$ 100%。

Benchmark	GT-judge	Self-judge	Retention
xBench	62.00	60.00	96.8%
GAIA	44.24	38.87	87.7%
WebWalkerQA	51.18	47.33	92.4%

偶尔错误标记技能的影响，使得即使没有真实反馈，结构信号仍保持可靠。

B.5 超图可视化

图 6 展示了图 5 中 xBench 超图子图在 WebWalkerQA 中的对应部分。

C 更广泛影响

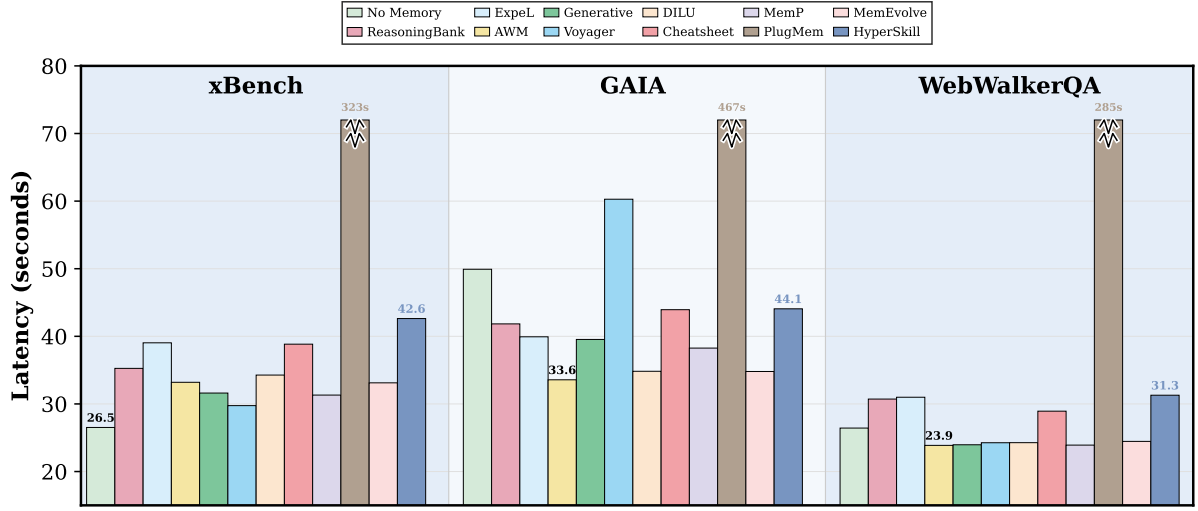
HYPERSKILL 是面向大语言模型智能体的通用记忆框架，不针对任何特定高风险应用。通过使智能体能够从过去的失败中学习，该框架有望减少冗余计算并提升部署系统的效率。然而，与任何自进化系统一样，如果不定期审计，累积的记忆可能会强化早期轨迹中存在的偏差。我们鼓励实践者监控已存储的技能，以发现非预期模式。

D 大语言模型使用披露

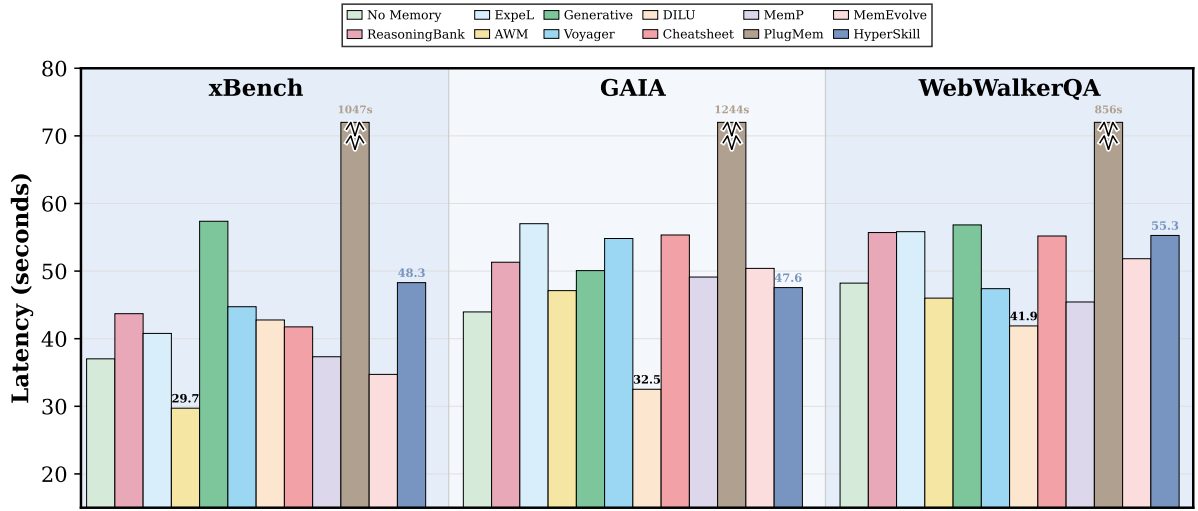
在本手稿准备过程中，基于大语言模型的工具被用于辅助语法修正、代码调试和图表设计。所有科学内容、实验设计和分析均由作者完成，作者对最终手稿承担全部责任。

E 扩展相关工作

大语言模型智能体系统。大语言模型正越来越多地被部署为自主智能体，通过与复杂环境进行多步推理和行动来交互，超越了传统的一次性聊天机器人范式 (Liu et al., 2025a; Xi et al., 2025; Wei et al., 2026)。一个核心挑战是使智能体能够将复杂任务分解为可管理的子问题并做出顺序决策。ReAct (Yao et al., 2022) 将思维链推理与环境行动交织在一起，为单智能体规划建立了一个基础范式。后续工作通过分层规划改进了任务分解 (Wang et al., 2023b; Sun et al., 2023)，并通过自我反思提高了决策质量 (Shinn et al., 2023)。多智能体系统 (Li et al., 2023; Wu et al., 2024; Talebirad and Nadiri, 2023) 通过为多个智能体分配专门角色，使其进行通信和协调，进一步扩展了这一范式，从而实现了超出单个智能体范围的协作问题解决。尽管这些进展显著拓宽了大语言模型智能体的能力边界，但大多数系统将每个任务片段视为孤立处理，限制了它们在重复出现或组合相关的任务模式上随时间改进的能力。智能体系统中的记忆机制。我们沿着三个维度组织现有的经验记忆系统：存储什么、记忆如何被结构化和检索、以及记忆如何演化。表 3 总结了比较结果。在存储内容方面，早期系统保留原始轨迹或从每个轨迹中提取洞见 (Zhao et al., 2024; Wen et al., 2024; Wang et al., 2023a)。后续工作将经验抽象为更紧凑的形式：工作流 (Wang et al., 2025b)、推理策略 (Ouyang et al.,



(a) Qwen3-30B-A3B.

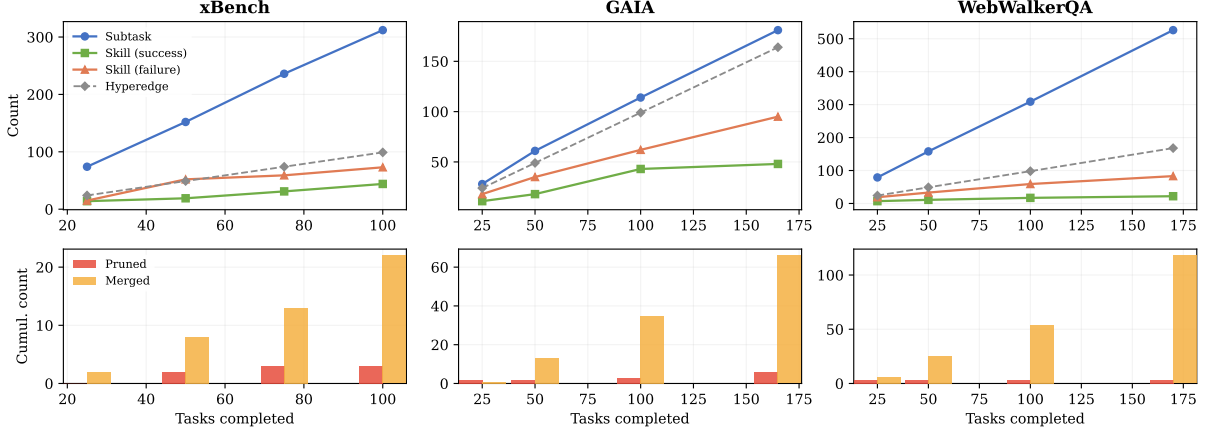


(b) GPT-4o.

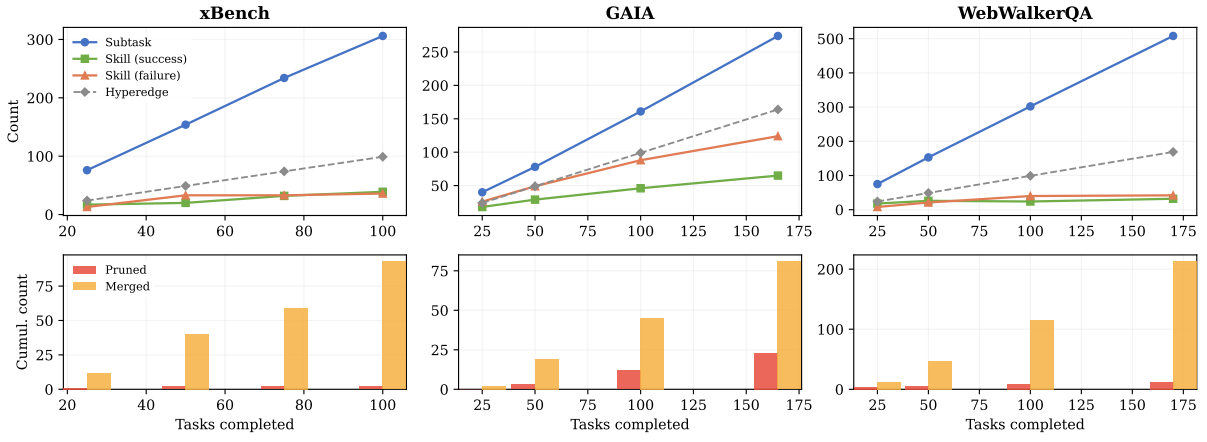
图 8: 在 xBench、GAIA 和 WebWalkerQA 上每个任务的平均墙钟延迟 (秒)。越低越好。标注的条形标记了每个面板中最快的基线, 以及 HYPERSKILL 的延迟作为参考。

2026)、可执行的应用程序编程接口技能 (Zheng 等, 2025), 或技巧与捷径 (Suzgun 等, 2026; Wang 等, 2025a)。近期基于训练的方法更进一步: 技能强化学习 (SkillRL) (Xia 等, 2026) 将轨迹蒸馏为分层技能库, 并通过强化学习 (群体相对策略优化, GRPO) 使其与智能体策略协同演化; 而记忆技能 (MemSkill) (Zhang 等, 2026) 将记忆操作本身视为可学习技能, 其选择通过强化学习优化, 其技能库通过分析困难案例演化。然而, 所有这些系统均孤立地存储知识单元, 丢弃了每条轨迹中子任务与技能之间的组合关系。超技能 (HYPERSKILL) 是唯一保留这种联合组合结构的系统。

关于记忆的结构化与检索方式, 大多数系统采用带语义检索的扁平向量存储 (Wang 等, 2023a; Wen 等, 2024; Shang 等, 2025), 少数采用对比检索, 比较成功与失败案例 (Zhao 等, 2024; Ouyang 等, 2026)。自适应工作记忆 (AWM) (Wang 等, 2025b) 将所有条目拼接进提示, 技能编织者 (SkillWeaver) (Zheng 等, 2025) 使用函数签名匹配。基于图的系统, 如图记忆 (G-Memory) (Zhang 等, 2025a) 和智能体知识库 (Agent-KB) (Tang 等, 2025b), 引入成对边, 但将 n 元轨迹上下文分解为不连通的二元链接。超技能使用超边保留完整轨迹上下文, 并通过结合子任务级与任务级查询的双路径机制进行检索。



(a) Qwen3-30B-A3B



(b) GPT-4o

图 9: 各基准上的超图记忆增长 (上行) 与累积维护操作 (下行)。

该机制结合了子任务级与任务级查询。

关于记忆的演化方式，大多数系统无任何策展地累积知识 (Wang 等, 2023a; Zhao 等, 2024; Wen 等, 2024; Ouyang 等, 2026; Wang 等, 2025b)。少数确实维护记忆的系统应用轻量级操作，如测试与剪枝 (Zheng 等, 2025)、情景巩固 (Zhang 等, 2025a)、去重 (Tang 等, 2025b) 或失败驱动调整 (Fang 等, 2026)，每种操作均独立应用于每个节点，不考虑记忆拓扑。技能强化学习 (Xia 等, 2026) 与记忆技能 (Zhang 等, 2026) 通过强化学习训练循环演化其技能库，但这种演化作用于模型权重而非显式记忆结构。超技能执行结构感知维护，通过质量加权超图传播联合考虑节点效用与结构邻近性。

我们还在表 4 中专门比较了图结构记忆系统。大多数基于图的方法针对事实记忆，将环境或用户知识记录在知识图谱 (Knowledge Graph) 中 (Gutiérrez 等, 2024; Chhikara 等, 2025)、时序图 (Temporal Graph) 中 (Rasmussen 等, 2025)、层次图 (Hierarchical Graph) 中 (Wu 等, 2026) 或多视图图 (Multiview Graph) 中 (Jiang 等, 2026)。HyperMem (Yue 等, 2026) 使用三层超图 (主题、情节、事实) 进行由粗到细的检索，但针对的是事实性对话记忆而非经验性任务知识，并且不执行结构感知的维护。在经验记忆系统中，PlugMem (Yang 等, 2026) 和 G-Memory (Zhang 等, 2025a) 使用成对边，丢失了联合轨迹上下文；Agent-KB (Tang 等, 2025b) 结合了词汇和语义索引，但没有在技能上建立显式图结构。HYPERSKILL 是第一个使用 n 元超边的经验记忆系统，

表 3: 面向大语言模型智能体的经验记忆系统分类。轨迹=原始轨迹；策略=推理策略；工具库=可调用函数注册表；技能库=层次化技能库。 ‡ 基于训练的方法，通过强化学习将技能内化到模型权重中。

Method	What is Stored		How Structured & Retrieved		How Evolved
	Knowledge Form	Compositional?	Structure	Retrieval	Maintenance
Voyager (Wang et al., 2023a)	Traj. & Tips	✗	Vector DB	Semantic	None
ExpeL (Zhao et al., 2024)	Traj. & Insights	✗	Vector DB	Contrastive	None
Generative (Shang et al., 2025)	Traj. & Insights	✗	Vector DB	Semantic	None
DILU (Wen et al., 2024)	Traj.	✗	Vector DB	Semantic	None
AWM (Wang et al., 2025b)	Workflows	✗	In-Prompt	All-in-Prompt	None
Mobile-E (Wang et al., 2025a)	Tips & Shortcuts	✗	Vector DB	Semantic	None
Cheatsheet (Suzgun et al., 2026)	Tips & Shortcuts	✗	JSON	Semantic	None
ReasoningBank (Ouyang et al., 2026)	Reasoning Strat.	✗	Vector DB	Contrastive	None
SkillWeaver (Zheng et al., 2025)	APIs	✗	Tool Lib.	Function Match	Test & Prune
G-Memory (Zhang et al., 2025a)	Tips & Workflows	✗	Graph	Graph + Semantic	Consolidation
Agent-KB (Tang et al., 2025b)	Tips & Workflows	✗	Hybrid DB	Hybrid	Deduplication
Memp (Fang et al., 2026)	Tips & Workflows	✗	JSON	Semantic	Failure-driven
EvolveR (Wu et al., 2025b)	Tips & Workflows	✗	JSON	Contrastive	Update & Pruning
SkillRL [‡] (Xia et al., 2026)	Hierarchical Skills	✗	Markdown	Adaptive	RL Co-Evolution
MemSkill [‡] (Zhang et al., 2026)	Learnable Skills	✗	Markdown	RL-guided	Skill Evolution
HYPERSKILL	Workflows & Skills	✓	Hypergraph	Dual-path	Quality + Topology

表 4: 面向大语言模型智能体的图结构记忆系统。“记忆类型”对主要功能进行分类：事实型记录环境或用户知识；经验型从智能体轨迹中积累可复用的任务知识。“边类型”区分成对（二元）边与 n 元边（连接任意节点集的超边）。† Agent-KB 在不同框架间共享知识，但由单个智能体执行。

Method	Memory Scope		Graph Structure			Evolution
	Mem. Type	MAS	Graph Type	Edge	Retrieval	Maintenance
HippoRAG (Gutiérrez et al., 2024)	Factual	✗	KG	Pairwise	PPR Activation	None
AriGraph (Anokhin et al., 2025)	Factual	✗	KG + Episodic	Pairwise	Semantic + Episodic	Graph Update
A-Mem (Xu et al., 2026)	Factual	✗	Zettelkasten	Pairwise	Semantic + Linking	Self-Organizing
Mem0g (Chhikara et al., 2025)	Factual	✗	KG + Vector	Pairwise	Hybrid	Deduplication
Zep (Rasmussen et al., 2025)	Factual	✗	Temporal KG	Pairwise	Temporal + Semantic	Temporal Inval.
GAM (Wu et al., 2026)	Factual	✗	Hierarchical	Pairwise	Multi-factor Trav.	Event-driven Consol.
MAGMA (Jiang et al., 2026)	Factual	✗	Multi-Graph	Pairwise	Policy-guided Trav.	Async. Consol.
HyperMem (Yue et al., 2026)	Factual	✗	Hypergraph	n -ary	Coarse-to-fine	None
HyperGraphRAG (Luo et al., 2025)	Factual	✗	Hypergraph	n -ary	Hypergraph Retrieval	None
PlugMem (Yang et al., 2026)	Experiential	✗	Dual KG	Pairwise	Subgraph Retrieval	None
G-Memory (Zhang et al., 2025a)	Experiential	✓	Hierarchical	Pairwise	Graph + Semantic	Episodic Consol.
Agent-KB [†] (Tang et al., 2025b)	Experiential	✗	Hybrid	Pairwise	Hybrid	Deduplication
HYPERSKILL	Experiential	✗	Hypergraph	n -ary	Dual-path + κ	Quality + Topology

边，从而支持基于共现的技能排序和拓扑感知的维护，而成对设计无法支持这些功能。

F 伦理声明

我们提供了全面的方法学细节，包括超参数、提示词和算法伪代码，以确保完全可复现。我们的框架在公开基准（xBench、GAIA、WebWalkerQA）上进行评估，不收集或处理个人信息。基于大语言模型的工具仅用于语法修正、代码调试和图表设计；所有科学内容、实验设计和分析均由作者完成。

由作者完成。虽然超技能（HyperSkill）被设计为面向良性研究的通用记忆框架，但我们承认，如果不定期审计，累积的记忆可能会强化早期轨迹中存在的偏差，因此我们鼓励从业者监控存储的技能，以发现非预期的模式。

G 补充案例研究

为了说明超技能（HYPERSKILL）的检索与演化机制在实际中如何运作，我们在图 10 中展示了一个具有代表性的通用人工智能助手（GAIA）任务以及记忆维护的快照。我们还呈现了三个额外的端到端案例研究，涵盖三个基准。每个案例报告了任务、用于驱动细粒度检索的分解、从超图中浮现的技能与错误、智能体的逐步执行轨迹以及最终结果。它们共同说明了超技能的双路径检索如何转化为跨异构领域的具体行动改进。

Case 1: xBench — Literary Trivia

任务（译自中文）：一部 19 世纪的法国小说在出版时引发了道德丑闻和诉讼。在 1991 年的电影改编中，女主角坚持在自杀场景中使用什么口味的道具毒药？

标准答案： bitter-almond flavor.

Plan Decomposition

1. search_literary_work: identify the 19th-century French novel via scandal keywords.
2. find_movie_adaptations: locate the 1991 film adaptation and lead actress.
3. verify_prop_taste: crawl film trivia sources for the specific flavor detail.
4. confirm_taste_detail: validate the exact flavor description.

Retrieved Memory

过往经验（提炼的教训）：

✓ *When verifying historical details, prioritize authoritative sources and cross-verify claims across multiple trusted references.*

✗ 永远不要假设公开摘要中会提供详细数据；始终使用精确的、以实体为中心的查询来定位权威来源。

Skills:

• **多阶段验证：**将复杂的跨领域问题分解为顺序验证步骤。

• **直接来源确认：**

• **来源深度解析：**

pets.

应避免的错误：

• **过度依赖网络搜索：**反复进行一般性检索而未核实权威来源。

Agent Execution (10 steps)

1 - 2 网络搜索 → 识别出古斯塔夫·福楼拜的《包法利夫人》；初步毒药查询。
3 - 5 确认 克劳德·夏布洛尔 1991 年电影，伊莎贝尔·于佩尔饰演艾玛·包法利。
6 - 8 渐进式细化：跨多个来源搜索幕后花絮。
9 - 10 最终验证：从权威电影数据库中提取苦杏仁味细节。

Outcome

✓ **SUCCESS** — the lead actress insisted on using a bitter-almond-flavored prop poison to authentically replicate the taste of cyanide.

记忆如何帮助：多阶段验证技能指导了跨领域链的分解（文学 → 电影 → 幕后花絮），而对网络搜索的过度依赖错误模式防止了在通用查询上的循环。检索到的关于在可信参考文献之间交叉验证的经验强化了智能体从聚合器片段转向权威电影数据库的行动。

Case 2: GAIA — Scientific Calculation

Task

在奥马尔·瓦伦西亚-门德斯 2017 年论文中记录的小丑虾总长度的整数百分比是多少，该长度被喂给了 G.柯特·菲德勒 2002 年论文中同类型虾的海星？

Golden answer: 22.

Plan Decomposition

1. inspect_file_as_text: read the attached file for data from both papers.
2. python_code: calculate the integer-rounded percentage.

Retrieved Memory

过往经验（提炼的教训）：

✓ 当从学术论文中寻求精确数值时，优先通过带有作者姓名和出版年份的定向搜索直接访问原始来源。

✗ 绝不在未首先诊断系统级错误的情况下重试工具调用；在重新尝试之前始终验证基础设施就绪状态。

Skills:

• **领域特定查询细化**：在技术数据查询中包含领域特定术语和可靠来源过滤器。

• **单位转换**：

应避免的错误：

• **上下文细化不足**：未能将任务分解为可验证的子问题会导致非结构化搜索。

Agent Execution (7 steps)

1 - 2 对两篇论文并行执行 web_search；通过牛津学术对 Fiedler 2002 执行 crawl_page。第 3 - 4 步使用具体测量值细化查询；直接阅读 Valencia-Mendez **2017** 的 PDF。第 5 - 6 步交叉引用：虾的总长度 $\boxed{= 4.5}$ 厘米，海星碎片 $\boxed{= 1 \text{ cm}}$ 。 **7** final_answer: 四舍五入 ($1 / 4.5 \times 100 = 22\%$)。

Outcome

✓ **SUCCESS** — the sea star was approximately 22% of the harlequin shrimp's total length.

记忆如何提供帮助：领域特定查询细化促使智能体使用精确的科学术语（“总长度”、“小丑虾”）而非泛化查询。上下文细化不足的教训警示了非结构化搜索，引导智能体将任务拆分为两个并行的数据检索子任务（每篇论文一个），然后再计算比率。

Case 3: WebWalkerQA — Game Knowledge

任务（译自中文）：在《帝国时代 IV》中，马里文明与中国文明在各自的第四时代有哪些战略差异？

标准答案：马里人利用格里奥特巴拉地标获得全局增益，而中国人则依赖长城作为防御优势。

Plan Decomposition

1. explore_site: crawl ageofempires.com homepage for site structure.
2. navigate_section: crawl the civilizations section for Malian and Chinese details.
3. find_target: search with domain-specific keywords on the official site.

Retrieved Memory

过往经验（提炼的教训）：

✓ *When institutional websites fail to surface specific records, precise multi-source searches are essential for retrieving accurate information.*

✗ 依赖自动化爬取而不验证访问权限或适应站点特定障碍会导致数据检索失败。

✗ 在充分浏览内部网站之前就进行外部搜索，可能会遗漏结构化且未被索引的信息。

Skills:

- **内部优先策略：**在诉诸外部搜索工具之前，彻底探索内部站点结构。
- **具体查询细化：**在搜索内部或外部工具时，使用精确的、领域特定的关键词。
- **利用多模态来源进行验证：**当内部搜索结果有限时，使用外部来源收集补充信息。

应避免的错误：

- **过早的外部搜索：**在彻底探索内部站点导航之前依赖外部网络搜索。
- **内部爬取不完整：**

te.

Agent Execution (11 steps)

1 - 2 `crawl_page(ageofempires.com)` → 探索主页；`web_search` 获取一般信息。3 - 5 迭代爬取官方网站的文明板块。6 - 8 查阅社区来源（Reddit、YouTube）以获取第四时代的具体信息。9 - 11 `crawl_page` 在 Fandom 维基上针对马里人和中国人 → 提取完整细节。

Outcome

✓成功——详细比较：马里人倾向于以格里奥特巴拉地标为核心的经济游击战术；中国人则依赖快速建造、长城防御和化学技术。

记忆如何提供帮助：内部优先策略技能使智能体不会立即转向外部搜索，而是先探索官方网站结构。三次失败教训都强化了同一模式：先内部导航，再转向外部。这是从之前 6 个以上涉及机构网站的 WebWalkerQA 任务中学到的模式，现已成功迁移到游戏维基。

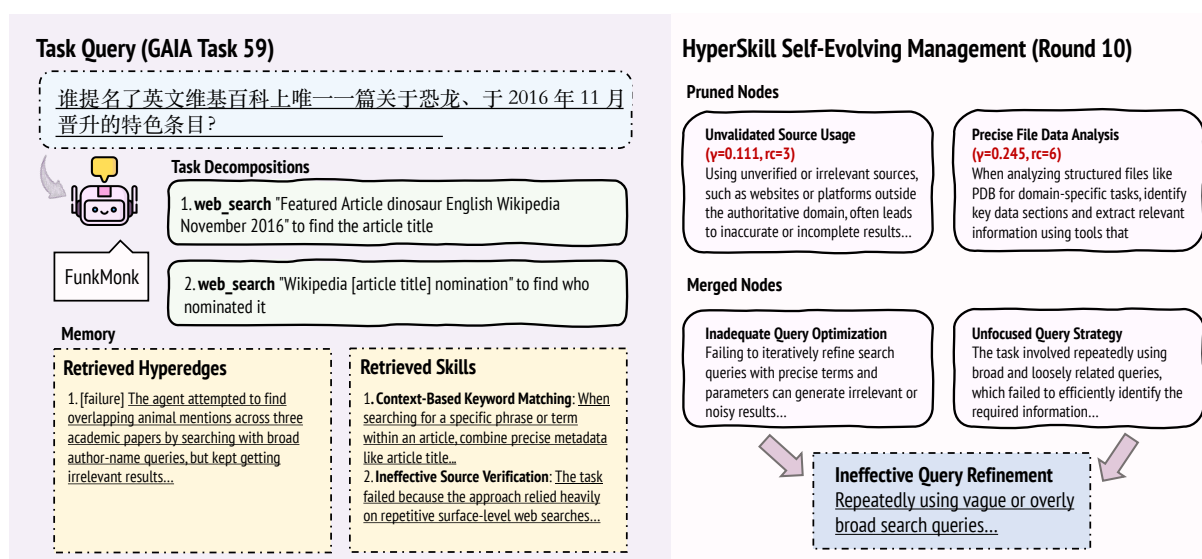


图 10: GAIA 任务 59 的案例研究。

H 算法

算法 1 总结了每个任务的完整 HYPERSKILL 流程，算法 2 描述了周期性的记忆演化。

Algorithm 1 HYPERSKILL: Per-Task Pipeline

Require: Task d_q ; memory $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; budgets k_u, k_e, k_s ; dedup threshold δ_{dedup}

Ensure: Updated memory $\mathcal{G}$

```

    % Dual-path hyperedge retrieval (§3.3)
    1:  $\mathcal{P}_0 \leftarrow \text{LLM-DECOMPOSE}(d_q)$  ▷ Subtask decomposition
    2:  $R_u \leftarrow \text{top-}k_u \text{ subtask nodes by } \text{sim}(\phi(\mathcal{P}_0), \phi(c_v))$  ▷ Eq. 4
    3:  $\mathcal{E}_{\text{sub}} \leftarrow \{e \in \mathcal{E} \mid V_e \cap R_u \neq \emptyset\}$  ▷ Subtask path
    4:  $\mathcal{E}_{\text{traj}} \leftarrow \text{top-}k_e \text{ hyperedges by } \text{sim}(\phi(d_q), \mathbf{h}_e)$  ▷ Trajectory path ▷ Eq. 5
    5:  $\mathcal{E}^* \leftarrow \mathcal{E}_{\text{sub}} \cup \mathcal{E}_{\text{traj}}$  ▷ Fuse ▷ Eq. 6
    % Skill ranking & execution (§3.4)
    6:  $R_s \leftarrow \bigcup_{e \in \mathcal{E}^*} V_e^s$ ;  $\kappa(v) \leftarrow |\{e \in \mathcal{E}^* \mid v \in V_e\}|$  for each  $v \in R_s$  ▷ Eq. 8
    7:  $\mathcal{S}_q \leftarrow \text{top-}k_s \text{ skills by } \kappa(v)$ ;  $\mathcal{L}_q \leftarrow \{\ell_e \mid e \in \mathcal{E}^*\}$  ▷ Eq. 9
    8: Execute task with context  $(\mathcal{S}_q, \mathcal{L}_q)$ ; observe outcome  $r_i$ , steps  $T_i$  ▷ Eq. 10
    % Memory update
    9: Extract skills  $S_i^{\text{new}}$  and lesson  $\ell_{e_i}$  from trajectory  $\tau_i$ 
    10: Deduplicate: merge into existing skill if  $\text{sim} \geq \delta_{\text{dedup}}$ , else add new node
    11: Add hyperedge  $e_{\text{new}} = (\mathcal{P}_0 \cup S_i^{\text{new}}, d_q, \ell_{e_i})$  to  $\mathcal{E}$ 
    12: Update utility  $\gamma(\cdot)$  via Eq. 3 for all retrieved elements

```

Algorithm 2 HYPERSKILL: Periodic Memory Evolution (every $\lceil 0.1 \times N \rceil$ tasks)

Require: Memory $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; thresholds $\tau_{\text{prune}}, N_{\text{min}}, \delta_{\text{merge}}$; propagation depth L

```

    % Quality-driven pruning (§3.5)
    1: Remove all  $v$  with  $\nu(v) \geq N_{\text{min}}$  and  $\gamma(v) < \tau_{\text{prune}}$  from  $\mathcal{V}$  and incident hyperedges ▷ Eq. 11
    % Structure-informed merging (skill nodes  $\mathcal{V}_s$  only)
    2:  $W_{ij} \leftarrow \min(\gamma(v_i), \gamma(v_j)) \cdot \sum_{v_i \in V_e, v_j \in V_e} |V_e|^{-1}$  ▷ Quality-weighted co-occurrence ▷ Eq. 12
    3:  $\tilde{\mathbf{Z}} \leftarrow \mathbf{S} \mathbf{Z}$  via  $L$ -step propagation ▷ Structural embeddings ▷ Eq. 13
    4: for each pair  $(v_a, v_b)$  with  $\text{sim}(\tilde{\mathbf{z}}_a, \tilde{\mathbf{z}}_b) \geq \delta_{\text{merge}}$  do
    5:    $v_{\text{new}} \leftarrow \text{LLM-MERGE}(v_a, v_b)$ ; reassign hyperedge memberships ▷ Eq. 14
    6: end for

```

一、提示词

Task Decomposition Prompt (GAIA)

Decompose this task into 1–3 SHORT steps. Minimize steps to save context window.

Available tools:

- `web_search(query)`: Search with SPECIFIC keywords (names, dates, exact phrases)
- `crawl_page(url)`: Read a URL — use ONLY when you need specific page content
- `inspect_file_as_text(path)`: Read attached files — START HERE if task has a file
- `inspect_file_as_image(path)`: View images/figures
- `python code`: Calculations, counting, data processing

Key principles:

- If a file is attached, read it FIRST
- Use `web_search` with the MOST specific terms possible
- Prefer python code for calculations over searching

Task: {query}

Return ONLY a JSON array: [{"name": "action_name", "description": "use <tool> to <specific action>"}]

Task Decomposition Prompt (WebWalkerQA)

将此网页导航任务分解为 2 至 4 个步骤。可用工具：

- `web_search(query)`: Search for specific pages on the website
- `crawl_page(url)`: Read content from a URL

导航策略：始终先抓取根网址以了解网站结构。然后导航至相关板块（新闻、奖项、教师等），再搜索具体信息。示例：“在 university.edu 上查找 X 教授 2021 年获得的奖项”：

```
[{"name": "explore_site",
  "description": "crawl_page(root_url)
  to find navigation menu"},
{"name": "navigate_section",
  "description":
    "crawl_page(root_url/awards)"},
{"name": "find_target",
  "description":
    "web_search('Prof X award 2021
    site:university.edu')"}]
```

Task: {query}

Return ONLY a JSON array.

Task Decomposition Prompt (xBench)

Decompose this data task into 2–4 steps. Each step must specify WHICH DATA to get and HOW to process it.

Available tools:

- `web_search(query)`: Search for data sources
- `crawl_page(url)`: Read data from specific URLs
- `python code`: Run calculations, unit conversions, comparisons

Task: {query}

Return ONLY a JSON array: [{"name": "action_name", "description": "use <tool> to <specific data action>"}]

Skill Extraction Prompt (Success)

Extract reusable knowledge from this SUCCESSFUL task.

CRITICAL RULES:

- Names must be SHORT reusable pattern names (3–5 words), NOT task-specific. Bad: “Search for Zip Codes”. Good: “Targeted Database Search”.
- Content must be a COMPLETE paragraph (2–4 sentences) that covers: WHAT to do, WHEN to apply it, and WHY it works.
- Extract 1–3 items max. Quality over quantity.
- Do NOT include task-specific details (exact names, IDs, URLs). Make knowledge transferable.

Task: {query}

Trajectory: {trajectory}

Result: {result}

Extract skills (what strategies worked well).

Extract as JSON:

```
{
  "skills": [
    {
      "name": "Short Skill Name",
      "content":
        "2-4 sentence paragraph..."
    },
    {
      "name": "Short Skill Name",
      "content":
        "2-4 sentence paragraph..."
    }
  ],
  "knowledge_fragment":
    "ONE OR TWO sentences stating the
    core transferable lesson."
}
```

Mistake Extraction Prompt (Failure)

Extract reusable lessons from this FAILED task.

CRITICAL RULES: (same as success extraction)

Task: {query}

Trajectory: {trajectory}

Result: {result}

Extract mistakes (what went wrong and how to avoid it).

Extract as JSON:

```
{
  "mistakes": [
    {
      "name": "Short Mistake Name",
      "content":
        "2-4 sentence paragraph
        describing what fails, why, and
        what to do instead."
    },
    {
      "name": "Short Mistake Name",
      "content":
        "2-4 sentence paragraph
        describing what fails, why, and
        what to do instead."
    }
  ],
  "knowledge_fragment":
    "ONE OR TWO sentences
    stating the core failure mode to avoid."
}
```

Structure-Informed Merging Prompt

These two {type}s are structurally and semantically related. Consolidate them into ONE higher-order {type} that captures the knowledge from both.

{TYPE} A:

Name: {name_a}

Content: {content_a}

{TYPE} B:

Name: {name_b}

Content: {content_b}

Create a consolidated {type} that:

- Has a SHORT reusable name (3–5 words)
- Has content (2–4 sentences) covering BOTH strategies/patterns
- Is more general and widely applicable than either individual {type}

Reply with ONLY JSON: {"name": "...", "content": "..."}"

Retrieved Memory Format (Agent Context)

以下记忆在每个回合开始时组装一次，并在执行开始时提供给智能体：## 过往经验（从排名前 k 的检索超边中提炼的经验）

1. [success] When official exchange data is rate-limited, cross-verified third-party aggregators provide a reliable fallback...
2. [failure] Relying on generic search terms instead of querying domain-specific databases yields incomplete data...

Relevant Skills

1. **Targeted Database Search:** Search for specific financial metrics using precise query terms that include the asset, time frame, and required data points...

Mistakes to Avoid

1. **Overreliance on Unverified Aggregators:** Using third-party summaries or social media posts as primary sources for quantitative claims leads to incorrect results...